



Mitigating Non-Target Speaker Bias in Guided Speaker Embedding

Shota Horiguchi, Takanori Ashihara, Marc Delcroix, Atsushi Ando, Naohiro Tawara

NTT, Inc., Japan
horiguchi@ieee.org

Abstract

Obtaining high-quality speaker embeddings in multi-speaker conditions is crucial for many applications. A recently proposed guided speaker embedding framework, which utilizes speech activities of target and non-target speakers as clues, drastically improved embeddings under severe overlap with small degradation in low-overlap cases. However, since extreme overlaps are rare in natural conversations, this degradation cannot be overlooked. This paper first reveals that the degradation is caused by the global-statistics-based modules, widely used in speaker embedding extractors, being overly sensitive to intervals containing only non-target speakers. As a countermeasure, we propose an extension of such modules that exploit the target speaker activity clues, to compute statistics from intervals where the target is active. The proposed method improves speaker verification performance in both low and high overlap ratios, and diarization performance on multiple datasets.

Index Terms: speaker embedding, speaker verification, speaker diarization

1. Introduction

In long-form multi-speaker audio processing, a commonly used approach involves processing short segments to obtain results with intra-segment speaker labels and speech activity intervals, followed by aligning speakers across segments. For example, some speaker diarization methods first estimate speaker-wise speech activities within a sliding window and then estimate which pair of speakers across windows are the same identity [1–3]. The same can also be done for speech recognition methods that provide overlap-aware multi-speaker recognition results along with intra-segment speaker labels and timestamps [4–9]. This two-stage approach has the advantage of eliminating the need to develop models that can handle an infinite number of speakers and input length.

Speaker embeddings are crucial for aligning speakers across segments, as they determine whether speakers across segments are the same or different based on their similarities. Here, it is important that the speaker embeddings do not get contaminated by interference speakers. One commonly used approach is to compute the embedding using only intervals where each speaker is speaking alone [1–3, 10, 11]. However, it may result in low-quality embeddings when the single-speaker intervals are too short or even not available.¹ Another approach is to use speech separation or extraction to remove interference speakers beforehand [12], but they usually cause artifacts and distortion that may affect the quality of the embeddings. To

¹In such cases, speaker embeddings are often extracted from overlapping segments out of necessity [2].

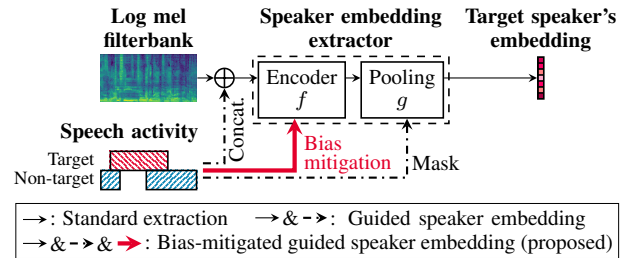


Figure 1: Block diagram of standard single-speaker embedding extraction, guided speaker embedding extraction, and the proposed bias-mitigated guided speaker embedding extraction.

address these limitations, guided speaker embedding has been recently proposed to extract speaker embeddings directly from multi-speaker recordings by leveraging speech activities of target and non-target speakers [13]. This is a promising direction as it enables speaker embedding extraction from overlapping segments without relying on separation or enhancement. However, the performance in situations with less overlap is slightly compromised. Since the typical overlap ratio in real-world conversations is not very high, there are cases where the significant improvement in highly overlapping situations cannot outweigh the degradation in less overlapping situations.

This paper first identifies the cause of the aforementioned performance degradation in guided speaker embedding. While non-target speaker intervals help identify the information to be removed in overlapping speech, they also introduce unnecessary bias, which becomes apparent when the overlap is minimal. We reveal that this bias arises in widely-used modules that use statistics computed over the entire input sequence, caused by intervals where only non-target speakers speak, contaminating these statistics. In the proposed method, we mitigate this bias by adapting the modules to use statistics calculated solely from the target speaker's intervals, instead of relying on global statistics as in Fig. 1. To demonstrate the broad effectiveness of the proposed method, experimental evaluation is performed using two different architectures (ECAPA-TDNN [14] and CAM++ [15]) on two different tasks (speaker verification and diarization).

2. Review of guided speaker embedding

2.1. Method overview

The guided speaker embedding framework enables the extraction of the target speaker's embedding, specified by speech activities of target and non-target speakers [13]. This section provides the formulation of the framework.

We first introduce a common speaker embedding extractor. Let $[x_t]_{t=1}^T$ denote a sequence of acoustic features $x_t \in \mathbb{R}^F$,

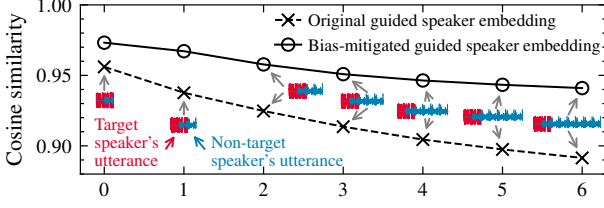


Figure 2: Cosine similarity between embeddings extracted using guided speaker embedding in a two-speaker case: one from a target speaker’s utterance and the other from a target/non-target mixture, with varying non-target-speaker-only durations.

where t is the time index. The processes inside the extractor to compute speaker embedding $\mathbf{v}$ are as follows:

$$[\mathbf{h}_t]_{t=1}^T = f([\mathbf{x}_t]_{t=1}^T) \in \mathbb{R}^{D \times T}, \quad (1)$$

$$\mathbf{v} = g([\mathbf{h}_t]_{t=1}^T) \in \mathbb{R}^E. \quad (2)$$

f is the encoder that transforms input features into frame-wise embeddings $[\mathbf{h}_t]_{t=1}^T$, and g is the pooling mechanism to aggregate the frame-wise embeddings into a single embedding followed by dimensionality reduction with a linear layer.

In guided speaker embedding, the following two modifications are made to the standard extractor as shown in Fig. 1. The first modification is that speech activities of target and non-target speakers are used for conditioning the input to the encoder, i.e., the following replaces the process in (1):

$$[\mathbf{h}_t]_{t=1}^T = f\left([\mathbf{x}_t \oplus [q_t^{\text{target}}, q_t^{\text{non-target}}]^T]_{t=1}^T\right) \in \mathbb{R}^{D \times T}, \quad (3)$$

where $\oplus$ denotes vector concatenation, q_t^{target} denotes the target speaker’s speech activity that takes 1 if the target speaker is active at t and 0 otherwise, and $q_t^{\text{non-target}}$ takes 1 if at least one non-target speaker is active at t and 0 otherwise. This enables enhancing the target speaker’s information while suppressing that of non-target speakers in the encoder. The second modification is to use only the embeddings of frames in which the target speaker is speaking for pooling:

$$\mathbf{v} = g([\mathbf{h}_t | q_t^{\text{target}} = 1]) \in \mathbb{R}^E. \quad (4)$$

Removing inactive intervals of the target speaker prevents contamination from non-target speakers in the output embedding.

2.2. Remaining problem

The important problem of guided speaker embedding is, as reported in the previous study, the degradation in less-overlapping conditions. The degree of degradation is indeed small (0.2 points in equal error rate, EER, in the single-speaker case [13]), but it causes a non-negligible performance drop in real applications, e.g., speaker diarization, since the overlap ratio of real conversations is typically not very high.

To reveal the reason for this, Fig. 2 shows an example of a two-speaker case: a target and a non-target speaker. We computed the cosine similarity between speaker embeddings extracted from a target speaker’s single-speaker utterance and its mixture with a non-target speaker’s utterance using the ECAPA-TDNN-based guided speaker embedding system [13]. We augmented the non-target-speaker-only duration by repeating the corresponding segment as in Fig. 2, where $m = 1$ is the original mixture, $m = 0$ removes non-target-speaker-only intervals, and $m > 1$ extends them m -fold. As in Sec. 2.1, guided speaker embedding is intentionally designed to convey no information about non-target speakers. However, lengthening the non-target speaker’s interval obviously affected the output embedding as

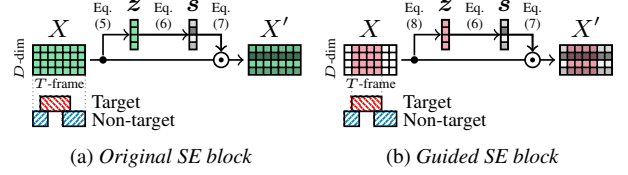


Figure 3: Original and proposed guided SE blocks. The colored cells in X are used for computing $\mathbf{z}$.

the cosine similarity decreased. The proposed method, explained in the next section, successfully mitigates the decrease in cosine similarity with increasing m , as shown in Fig. 2.

3. Proposed method

In this paper, we extend the conditioning introduced in guided speaker embedding to all modules computing global statistics. This is based on the hypothesis that the excessive influence of non-target speakers, which occurs not only during inference as in Sec. 2.2 but also during training, stems from modules using global statistics, and we propose using statistics calculated from the target speaker’s intervals instead. In particular, we focus on the ECAPA-TDNN [14] and CAM++ [15] architectures and describe how the proposed method is applied to squeeze-and-excitation (SE) block, context-aware masking, and batch normalization used therein. Note that the proposed method can be applied to other extractors that use these modules [16–18], those that employ other modules based on global statistics [19–21].

3.1. Guided squeeze-and-excitation block

SE block is a module to recalibrate channel-wise importance [22]. After being first proposed for computer vision models, it was introduced into speaker verification models and is now used in various architectures [14, 17, 23, 24]. In this paper, we focus on a one-dimensional variant of SE blocks, which is adopted in widely-used ECAPA-TDNN [14].

In ECAPA-TDNN, SE block is inserted just after each TDNN-Res2Net-TDNN chain. Let $X = [\mathbf{x}_t]_{t=1}^T \in \mathbb{R}^{D \times T}$ be an input T -length sequence of D -dimensional features. The squeeze operation is a global average pooling for each channel:

$$\mathbf{z} = \frac{1}{T} \sum_{t=1}^T \mathbf{x}_t. \quad (5)$$

The excitation operation is channel-wise weighting using two-stacked linear layers:

$$\mathbf{s} = \sigma(W_4 \delta(W_3 \mathbf{z} + \mathbf{b}_3) + \mathbf{b}_4) \in (0, 1)^D, \quad (6)$$

$$X' = X \odot \underbrace{[\mathbf{s}, \dots, \mathbf{s}]_T}_{T} \in \mathbb{R}^{D \times T}, \quad (7)$$

where $\{W_3 \in \mathbb{R}^{\frac{D}{r} \times D}, \mathbf{b}_3 \in \mathbb{R}^{\frac{D}{r}}\}$ and $\{W_4 \in \mathbb{R}^{D \times \frac{D}{r}}, \mathbf{b}_4 \in \mathbb{R}^D\}$ are the parameters of linear layers, $r \in \mathbb{R}_{>0}$ is the dimensionality reduction ratio, $\odot$ denotes element-wise product, and $\sigma(\cdot)$ and $\delta(\cdot)$ are sigmoid and rectified linear unit activations. These processes inside the SE block are depicted in Fig. 3(a).

The original SE block uses the entire sequence to compute channel-wise weights in (5)–(6). However, as shown in Fig. 2, the performance of the guided speaker embedding decreases as the duration of non-target speaker intervals increases. This is likely because non-target speaker intervals dominate the calculation of weights in an SE block, making it difficult to emphasize channels that are effective in extracting the target speaker’s information. To mitigate this, we propose calculating channel-wise weights using only frames in which the target speaker is

active $Q_{\text{target}} := \{t \mid q_t^{\text{target}} = 1\}$ as shown in Fig. 3(b). Specifically, the squeeze operation in (5) is replaced by

$$z = \frac{1}{|Q_{\text{target}}|} \sum_{t \in Q_{\text{target}}} x_t. \quad (8)$$

This allows the SE block to place more weight on channels that are important for extracting the target speaker's information.

3.2. Guided context-aware masking

Context-aware masking (CAM) can be seen as an extension of SE [25]. While the SE block directly changes the input X via channel-wise weighting, CAM performs input transformation $g : \mathbb{R}^{D \times T} \rightarrow \mathbb{R}^{D \times T}$ using TDNNs and channel-wise weight computation from the input by (6) in parallel. The output is calculated as their channel-wise product by replacing (7) with

$$X' = g(X) \odot \underbrace{[s, \dots, s]}_T \in \mathbb{R}^{D \times T}. \quad (9)$$

In this paper, we use the enhanced version of CAM, namely, CAM++, [15]. The original CAM calculates the weights using global statistics z , but local statistics are also used in CAM++. Specifically, a T -length input is split into K segments, where the indices of the k -th segment is $\{t_{k-1} + 1, t_{k-1} + 2, \dots, t_k\}$ with $t_0 = 0$ and $t_K = T$. Instead of (6) and (9), the weighting is based on not only z in (5) but also on the local statistics z_k :²

$$z_k = \frac{1}{t_k - t_{k-1}} \sum_{t=t_{k-1}+1}^{t_k} x_t, \quad (10)$$

$$s_k = \sigma(W_4 \delta(W_3(z + z_k) + b_3) + b_4) \in (0, 1)^D, \quad (11)$$

$$X' = g(X) \odot \underbrace{[s_1, \dots, s_1]}_{t_1} \underbrace{[s_2, \dots, s_2]}_{t_2 - t_1} \dots \underbrace{[s_K, \dots, s_K]}_{T - t_{K-1}}. \quad (12)$$

Since CAM++ also uses global statistics z in (11), it faces the same issue as the SE block, where the importance of channels is excessively biased toward non-target speakers. The proposed method, namely guided CAM++, replaces z with the one calculated only from the target speaker's intervals using (8).

3.3. Guided batch normalization

Batch normalization [26] smooths the loss function in deep neural networks, preventing vanishing or exploding gradients and enabling fast convergence with a large learning rate [27, 28]. Assume a mini-batch of input $\mathbf{X} = [X_b]_{b=1}^B$, where B is the batch size and $X_b = [x_{b,d,t}]_{d=1,t=1}^{D,T}$ denotes the b -th sample— T -length sequence of D -dimensional vectors. Batch normalization transforms each sample X_b into the same size of matrix $Y_b = [y_{b,d,t}]_{d=1,t=1}^{D,T}$ by

$$y_{b,d,t} = \gamma_d \frac{x_{b,d,t} - \mathbb{E}_d(\mathbf{X})}{\sqrt{\mathbb{V}_d(\mathbf{X}) + \varepsilon}} + \beta_d, \quad (13)$$

where $\mathbb{E}_d(\cdot)$ and $\mathbb{V}_d(\cdot)$ denote mean and variance of all the elements of the d -th dimension of input tensor, i.e.,

$$\mathbb{E}_d(\mathbf{X}) = \frac{1}{BT} \sum_{b=1}^B \sum_{t=1}^T x_{b,d,t}, \quad (14)$$

$$\mathbb{V}_d(\mathbf{X}) = \frac{1}{BT} \sum_{b=1}^B \sum_{t=1}^T (x_{b,d,t} - \mathbb{E}_d(\mathbf{X}))^2, \quad (15)$$

$\gamma = [\gamma_1, \dots, \gamma_D]^T \in \mathbb{R}^D$ and $\beta = [\beta_1, \dots, \beta_D]^T \in \mathbb{R}^D$ are learnable parameters, and $\varepsilon \ll 1$ is for numerical stability.

In guided speaker embedding, target-speaker intervals mainly contribute to the output speaker embedding. If batch

²We did not use the proposed guiding method for z_k since it is uncomputable in local segments where the target speaker is not speaking.

normalization is applied across all frames, mean and variance are also affected by those in which the target speaker is not active; this could unnecessarily bias the normalized results toward non-target speakers. We propose using the statistics calculated only from target-speaker intervals by replacing (14)–(15) with

$$\mathbb{E}_d(\mathbf{X}) = \frac{1}{B|Q_{\text{target}}|} \sum_{b=1}^B \sum_{t \in Q_{\text{target}}} x_{b,d,t}, \quad (16)$$

$$\mathbb{V}_d(\mathbf{X}) = \frac{1}{B|Q_{\text{target}}|} \sum_{b=1}^B \sum_{t \in Q_{\text{target}}} (x_{b,d,t} - \mathbb{E}_d(\mathbf{X}))^2. \quad (17)$$

Note that guided batch normalization is effective only during training and, during inference, uses the mean and variance computed during training, like standard batch normalization.

4. Experiments

4.1. Experimental setup

The following two encoders were used for the speaker embedding extractors: ECAPA-TDNN with 1024 channels [14] and CAM++ [15]. We used channel- and context-dependent attentive statistics pooling [14] to aggregate frame-wise embeddings from the encoders, followed by a linear layer to compute a 192-dimensional output. Each model takes 80-dimensional log mel filterbanks extracted with a 25 ms window and a 10 ms shift as input. In the case of guided speaker embedding, target and non-target speakers' speech activities are also input.

The models were trained using VoxCeleb1&2 development set [29]. The baseline single-speaker models were trained using utterances cropped to three seconds with a batch size of 256. The guided speaker embedding models were trained using 256 partially overlapped three-speaker mixtures, resulting in a batch size of 768. These mixtures are created on the fly by adding 2–4 second utterance clips, each shifted by at least 0.5 seconds. Each model was trained using additive angular margin softmax loss [30] with a margin of 0.2 and scale of 30. We use the Adam optimizer [31] for 80 epochs of training with cyclical learning rate scheduling. Each cycle lasts 20 epochs, warming up in the first 1 000 iterations and decaying in the rest using cosine annealing. On-the-fly data augmentation using noise [32] and room impulse responses [33] were applied during training.

When the proposed method is applied to ECAPA-TDNN, the batch normalization and SE block are replaced by the guided variants described in Sec. 3. In the case of CAM++, the batch normalization in dense blocks and context-aware masking are replaced by the corresponding variants,

4.2. Speaker verification results

Following the original guided speaker embedding study [13], we used VoxCeleb1-O for the standard speaker verification between single-speaker utterances (*one-vs-one*) and its extended versions for verifying whether the target speaker in a multi-speaker mixture is the same identity in a single-speaker utterance (*one-vs-many*). For the latter purpose, each trial in VoxCeleb1-O was extended by mixing three interference speakers' utterances, resulting in five evaluation sets with different overlap ratios of the target speaker's utterance: (0,25)%, [25,50)%, [50,75)%, [75,100)%, and 100%, i.e., fully overlapped. We report EERs assuming that oracle speaker-wise speech activities are available.

Table 1 shows the results of speaker verification. First, we focus on the results of ECAPA-TDNN. In line with the previous study [13], the original guided speaker embedding (B1-

Table 1: Speaker verification results in EERs (%). Single-speaker models (B1-1 & B2-1) used only single-speaker intervals for extraction. †/‡ denotes that P1-1 (P2-1) is significantly different from B1-1/2 (B2-1/2) with bootstrap method ($p < .05$, $n = 1000$).

		one-vs-one	one-vs-many				
ID	Method	0 %	(0,25) %	[25,50) %	[50,75) %	[75,100) %	100 %
Encoder: ECAPA-TDNN							
Baseline	B1-1 Single-speaker model [14]	0.88	1.29	1.75	2.59	14.37	27.64
	B1-2 Guided speaker embedding [13]	1.10	1.23	1.33	1.58	2.85	7.37
Proposed	P1-1 Bias-mitigated guided speaker embedding	0.93 [‡]	1.02 ^{†‡}	1.18 ^{†‡}	1.38 ^{†‡}	2.37 ^{†‡}	8.09 ^{†‡}
	P1-2 $\hookrightarrow$ w/o guided batch normalization	0.90	1.04	1.24	1.42	2.56	8.94
	P1-3 $\hookrightarrow$ w/o guided SE block	1.32	1.63	1.58	1.86	3.19	7.82
Encoder: CAM++							
Baseline	B2-1 Single-speaker model [15]	0.76	1.13	1.56	2.50	13.84	28.70
	B2-2 Guided speaker embedding [13]	0.93	1.03	1.15	1.44	2.66	7.68
Proposed	P2-1 Bias-mitigated guided speaker embedding	0.88 [†]	0.94 [†]	1.04 ^{†‡}	1.21 ^{†‡}	2.13 ^{†‡}	6.69 ^{†‡}
	P2-2 $\hookrightarrow$ w/o guided batch normalization	0.88	0.99	1.07	1.28	2.10	6.40
	P2-3 $\hookrightarrow$ w/o guided CAM++	0.90	1.07	1.19	1.38	2.59	7.23

2) improved EERs on the *one-vs-many* condition compared to the single-speaker model (B1-1), while degraded on the *one-vs-one* condition. The proposed bias-mitigated guided speaker embedding (P1-1) not only improved the EER in *one-vs-one* from 1.10 % to 0.93 %, but also enhanced the EER in all *one-vs-many* conditions except for the fully overlapped one. This is the expected behavior of the proposed method, which improves speaker discrimination in low-overlapping conditions. We also performed an ablation study by replacing guided batch normalization and SE blocks with their original versions in P1-2 and P1-3, respectively. Overall, both modules were found to contribute to the performance improvement, especially the significant improvement caused by bias mitigation in the SE block.

The same trend was observed in the case of CAM++. The original guided speaker embedding (B2-2) improved the EERs of single-speaker baseline (B2-1) in all the *one-vs-many* conditions while degraded in the *one-vs-one* condition. The proposed bias-mitigated guided speaker embedding (P2-1) successfully outperformed B2-2 in all the cases. In terms of each module, guided CAM++ is important for performance improvement in all cases, while guided batch normalization contributed to performance improvement in low-overlapping conditions.

To verify the proposed method’s effectiveness in mitigating non-target speaker bias, we analyzed EER transitions as non-target-speaker-only durations were scaled like in Fig. 2. As shown by each blue line in Fig. 4, increasing non-target speaker intervals degraded EER with the original guided speaker embedding (B2-2). One might assume that not using intervals where only non-target speakers speak (i.e., $m = 0$) would improve performance, but this significantly degrades EER when the overlap ratio is high ($\geq 75\%$). This is because information that should not be included in speaker embeddings, easily obtained from intervals containing only non-target speakers, cannot be identified and exploited. The proposed method eliminated this negative trend, achieving stable EER regardless of non-target-speaker-only duration (red lines in Fig. 4).

4.3. Speaker diarization results

Speaker diarization performance was evaluated on eight domains from six datasets: AISHELL-4 [34], AliMeeting [35], AMI Mix-Headset and the first channel of array [36], MSD-Wild [37] few- and many-talker sets, DIHARD III [38], and VoxConverse [39]. We used the pyannote.audio 3.1 [3] pipeline; it first performs local diarization with a sliding window of 10 s width and 1 s shift and then aggregates window-wise results by agglomerative hierarchical clustering of speaker embeddings.

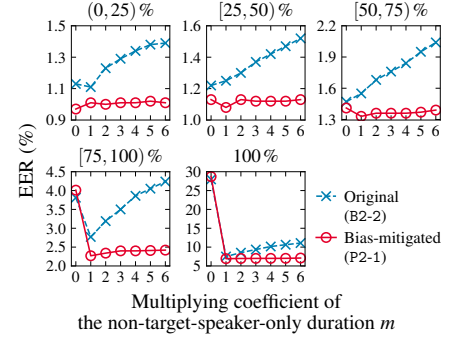


Figure 4: EERs with varied duration where the target speaker is not speaking. The multiplication coefficient $m = 1$ corresponds to the original one-vs-many evaluation set.

Table 2: Speaker diarization results in DERs (%) using speaker embedding extractors based on CAM++.

Dataset	Local diarization: pyannote.audio 3.1 [3]			Local diarization: Oracle		
	B2-1	B2-2	P2-1	B2-1	B2-2	P2-1
AISHELL-4 [34]	12.03	11.86	11.83	3.05	3.09	3.39
AliMeeting [35]	21.03	21.24	19.66	10.46	5.52	4.35
AMI Mix-Headset [36]	17.43	17.87	17.45	6.16	4.30	3.92
AMI Array channel 1 [36]	21.48	21.10	20.20	7.66	6.24	5.57
MSDWild (few) [37]	25.08	24.41	24.40	10.00	7.69	6.95
MSDWild (many) [37]	48.80	48.05	49.19	28.78	27.15	24.67
DIHARD III [38]	21.12	21.45	21.18	6.03	6.56	6.02
VoxConverse [39]	11.94	11.52	11.08	3.20	2.49	2.56
Macro average	22.36	22.19	21.87	9.42	7.88	7.18

We used the pretrained local diarization model, and the following three speaker embedding extractors based on CAM++ were compared: the single-speaker model (B2-1), the original guided speaker embedding model (B2-2), and the proposed bias-mitigated model (P2-1). Only the single-speaker intervals were used for extraction in B2-1, while all the speech intervals were used in B2-2 and P2-1. To purely compare the performance of the extractors, we also examined cases where the oracle local diarization results are available. Diarization error rate (DER) without collar tolerance was used for evaluation.

The results are shown in Table 2. Regardless of the local diarization method, the naive guided speaker embedding (B2-2) outperformed the single-speaker model overall, and the proposed bias-mitigated guided speaker embedding (P2-1) brought additional improvements. If we get better local diarization, the effect of the proposed approach becomes even larger; the gains achieved by the proposed method (P2-1) over the baselines (B2-1 and B2-2) with pyannote.audio local diarization (0.49 and 0.32 points) are smaller than those of the oracle local diarization (2.24 and 0.70 points). One possible reason is that the guide based on speech activities is not very effective when local diarization contains errors. Future work will address developing a method that is robust to such errors or that utilizes soft guidance instead of binary guidance.

5. Conclusion

In this paper, we revealed that the performance degradation of guided speaker embedding in low-overlapping conditions is due to bias introduced by non-target speaker intervals. We proposed a bias mitigation method as a countermeasure, in which statistics calculated only from the target speaker’s intervals were used instead of global statistics. Speaker verification and diarization results demonstrated the effectiveness of the proposed method.

6. References

- [1] K. Kinoshita, M. Delcroix, and N. Tawara, “Integrating end-to-end neural and clustering-based diarization: Getting the best of both worlds,” in *Proc. ICASSP*, 2021, pp. 7198–7202.
- [2] H. Bredin, “pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe,” in *Proc. Interspeech*, 2023, pp. 1983–1987.
- [3] A. Plaquet and H. Bredin, “Powerset multi-class cross entropy loss for neural speaker diarization,” in *Proc. Interspeech*, 2023, pp. 3222–3226.
- [4] L. Lu, N. Kanda, J. Li, and Y. Gong, “Streaming end-to-end multi-talker speech recognition,” *IEEE Signal Processing Letters*, vol. 28, pp. 803–807, 2021.
- [5] L. Lu, J. Li, and Y. Gong, “Endpoint detection for streaming end-to-end multi-talker ASR,” in *Proc. ICASSP*, 2022, pp. 7312–7316.
- [6] I. Sklyar, A. Piunova, X. Zheng, and Y. Liu, “Multi-turn RNN-T for streaming recognition of multi-party speech,” in *Proc. ICASSP*, 2022, pp. 8402–8406.
- [7] N. Kanda, J. Wu, Y. Wu, X. Xiao, Z. Meng, X. Wang, Y. Gaur, Z. Chen, J. Li, and T. Yoshioka, “Streaming multi-talker ASR with token-level serialized output training,” in *Proc. Interspeech*, 2022, pp. 3774–3778.
- [8] C. Cui, I. Sheikh, M. Sadeghi, and E. Vincent, “Improving speaker assignment in speaker-attributed ASR for real meeting applications,” in *Proc. Odyssey*, 2024, pp. 99–106.
- [9] T. Moriya, S. Horiguchi, M. Delcroix, R. Masumura, T. Ashihara, H. Sato, K. Matsuura, and M. Mimura, “Alignment-free training for transducer-based multi-talker ASR,” in *Proc. ICASSP*, 2025.
- [10] N. Kanda, S. Horiguchi, Y. Fujita, Y. Xue, K. Nagamatsu, and S. Watanabe, “Simultaneous speech recognition and speaker diarization for monaural dialogue recordings with target-speaker acoustic models,” in *Proc. ASRU*, 2019, pp. 31–38.
- [11] I. Medennikov, M. Korenevsky, T. Prisyach, Y. Khokhlov, M. Korenevskaya, I. Sorokin, T. Timofeeva, A. Mitrofanov, A. Andrusenko, I. Podluzhny, A. Laptev, and A. Romanenko, “Target-speaker voice activity detection: a novel approach for multi-speaker diarization in a dinner party scenario,” in *Proc. Interspeech*, 2020, pp. 274–278.
- [12] M. Delcroix, K. Zmolikova, T. Ochiai, K. Kinoshita, and T. Nakatani, “Speaker activity driven neural speech extraction,” in *Proc. ICASSP*, 2021, pp. 6099–6103.
- [13] S. Horiguchi, T. Moriya, A. Ando, T. Ashihara, H. Sato, N. Tawara, and M. Delcroix, “Guided speaker embedding,” in *Proc. ICASSP*, 2025.
- [14] B. Desplanques, J. Thienpondt, and K. Demuynck, “ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification,” in *Proc. Interspeech*, 2020, pp. 3830–3834.
- [15] H. Wang, S. Zheng, Y. Chen, L. Cheng, and Q. Chen, “CAM++: A fast and efficient network for speaker verification using context-aware masking,” in *Proc. Interspeech*, 2023, pp. 5301–5305.
- [16] N. R. Koluguri, T. Park, and B. Ginsburg, “TitaNet: Neural model for speaker representation with 1D depth-wise separable convolutions and global context,” in *Proc. ICASSP*, 2022, pp. 8102–8106.
- [17] Z. Zhao, Z. Li, W. Wang, and P. Zhang, “PCF: ECAPA-TDNN with progressive channel fusion for speaker verification,” in *Proc. ICASSP*, 2023.
- [18] B. Liu and Y. Qian, “ECAPA++: Fine-grained deep embedding learning for TDNN based speaker verification,” in *Proc. Interspeech*, 2023, pp. 3132–3136.
- [19] J. Thienpondt, B. Desplanques, and K. Demuynck, “Integrating frequency translational invariance in TDNNs and frequency positional information in 2D ResNets to enhance speaker verification,” in *Proc. Interspeech*, 2021, pp. 2302–2306.
- [20] T. Liu, R. K. Das, K. A. Lee, and H. Li, “MFA: TDNN with multi-scale frequency-channel attention for text-independent speaker verification with short utterances,” in *Proc. ICASSP*, 2022, pp. 7517–7521.
- [21] H.-J. Heo, U.-H. Shin, R. Lee, Y. Cheon, and H.-M. Park, “NeXt-TDNN: Modernizing multi-scale temporal convolution backbone for speaker verification,” in *Proc. ICASSP*, 2024, pp. 11 186–11 190.
- [22] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proc. CVPR*, 2018, pp. 7132–7141.
- [23] T. Zhou, Y. Zhao, and J. Wu, “ResNeXt and Res2Net structures for speaker verification,” in *Proc. SLT*, 2021, pp. 301–307.
- [24] R. Wang, Z. Wei, H. Duan, S. Ji, Y. Long, and Z. Hong, “EfficientTDNN: Efficient architecture search for speaker recognition,” *IEEE/ACM TASLP*, vol. 30, pp. 2267–2279, 2022.
- [25] Y.-Q. Yu, S. Zheng, H. Suo, Y. Lei, and W.-J. Li, “CAM: Context-aware masking for robust speaker verification,” in *Proc. ICASSP*, 2021, pp. 6703–6707.
- [26] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. ICML*, 2015, pp. 448–456.
- [27] N. Bjorck, C. P. Gomes, B. Selman, and K. Q. Weinberger, “Understanding batch normalization,” in *Proc. NeurIPS*, 2018, pp. 7694–7705.
- [28] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How does batch normalization help optimization?” in *Proc. NeurIPS*, 2018, pp. 2488–2498.
- [29] A. Nagrani, J. S. Chung, W. Xie, and A. Zisserman, “VoxCeleb: Large-scale speaker verification in the wild,” *Computer Speech & Language*, vol. 60, p. 101027, 2020.
- [30] J. Deng, J. Guo, J. Yang, N. Xue, I. Kotsia, and S. Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” *IEEE TPAMI*, vol. 44, no. 10, pp. 5962–5979, 2022.
- [31] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [32] D. Snyder, G. Chen, and D. Povey, “MUSAN: A music, speech, and noise corpus,” arXiv:1510.08484, 2015.
- [33] T. Ko, V. Peddinti, D. Povey, M. L. Seltzer, and S. Khudanpur, “A study on data augmentation of reverberant speech for robust speech recognition,” in *Proc. ICASSP*, 2017, pp. 5220–5224.
- [34] Y. Fu, L. Cheng, S. Lv, Y. Jv, Y. Kong, Z. Chen, Y. Hu, L. Xie, J. Wu, H. Bu, X. Xu, J. Du, and J. Chen, “AISHELL-4: An open source dataset for speech enhancement, separation, recognition and speaker diarization in conference scenario,” in *Proc. Interspeech*, 2021, pp. 3665–3669.
- [35] F. Yu, S. Zhang, Y. Fu, L. Xie, S. Zheng, Z. Du, W. Huang, P. Guo, Z. Yan, B. Ma, X. Xu, and H. Bu, “M2MeT: The ICASSP 2022 multi-channel multi-party meeting transcription challenge,” in *Proc. ICASSP*, 2022, pp. 6167–6171.
- [36] J. Carletta, “Unleashing the killer corpus: experiences in creating the multi-everything AMI Meeting Corpus,” *Language Resources and Evaluation*, vol. 41, no. 2, pp. 181–190, 2007.
- [37] T. Liu, S. Fan, X. Xiang, H. Song, S. Lin, J. Sun, T. Han, S. Chen, B. Yao, S. Liu, Y. Wu, Y. Qian, and K. Yu, “MSDWild: Multi-modal speaker diarization dataset in the wild,” in *Proc. Interspeech*, 2022, pp. 1476–1480.
- [38] N. Ryant, P. Singh, V. Krishnamohan, R. Varma, K. Church, C. Cieri, J. Du, S. Ganapathy, and M. Liberman, “The third DI-HARD diarization challenge,” in *Proc. Interspeech*, 2021, pp. 3570–3574.
- [39] J. S. Chung, J. Huh, A. Nagrani, T. Afouras, and A. Zisserman, “Spot the conversation: Speaker diarisation in the wild,” in *Proc. Interspeech*, 2020, pp. 299–303.